Internet of Things 37 (2026) 101932



Contents lists available at ScienceDirect

Internet of Things

journal homepage: www.elsevier.com/locate/iot





Autonomous pentesting using artificial intelligence: from the cybersecurity point-of-view

Rui Fernandes [a,b,c,*], Nuno Lopes [a,b], Joaquim Gonçalves [a,b], John Cosgrove [c]

[a] 2Ai - School of Technology, IPCA, Barcelos, Portugal
[b] LASI - Associate Laboratory of Intelligent Systems, Guimarães, Portugal
[c] TUS - Technological University of the Shannon, Limerick, Ireland

ARTICLE INFO

Keywords:
Artificial intelligence
Cybersecurity
Hacking
Autonomous pentesting
Penetration testing

ABSTRACT

Reinforcement Learning (RL) is emerging as a key approach for autonomous Penetration Testing, enabling agents to learn offensive actions through interaction with simulated networks. However, current RL environments differ widely in realism, attack coverage, and scalability, creating a significant gap between experimental performance and real-world applicability. This work evaluates the internal design of state-of-the-art RL Pentesting simulators, identifying limitations in their action models, MITRE ATT&CK coverage, emulation support, reward mechanisms, network scalability, and environment fidelity. Our contribution is a technical comparison that isolates which environments best support each feature for practical Autonomous Pentesting. Results show that no simulator leads in all area. NASimEmu, Caldera, and Cyberwheel provide the highest environment fidelity; Caldera, Cyberwheel, and CybORG offer the broadest MITRE ATT&CK coverage; and NASim, MulVAL, and CyberBattleSim achieve the best scalability through lightweight graph-based designs.

1. Introduction

The Vulnerability Assessment process is an ongoing concern nowadays. It is an essential task for security remediation as it helps organisations in patching vulnerabilities before hackers exploit them [1]. Vulnerability Assessment or Penetration Testing involves using Artificial Intelligence (AI) to hack with ethical purposes. This research examines how AI can realistically enhance the Penetration Testing process at a time when adversaries increasingly rely on automation and AI-based attacks. Traditional tools are becoming insufficient for the current infrastructures. Evaluating modern AI-driven systems, identifying their practical limitations, and understanding how they can integrate into real PT workflows has therefore become essential.

In the last five years, attacks have increased in number and complexity [2]. Attacks such as the SolarWinds Supply Chain that happened in 2020, where an attack affected numerous organisations, including the US government, are still being studied [3]. Besides traditional cyberattacks, new ones were launched, for example, the Deepfakes targeting the CEO or people involved in the Russia-Ukraine war [4].

With the new types of attacks, traditional Cybersecurity tools for vulnerability assessment aren't good enough. Today's world needs tools that use AI in their engines so that efficiency and performance can be improved. Automating existing tools aligned with

* Corresponding author.
E-mail addresses: rpfernandes@ipca.pt (R. Fernandes), nlopes@ipca.pt (N. Lopes), jgoncalves@ipca.pt (J. Gonçalves), John.Cosgrove@tus.ie (J. Cosgrove).

https://doi.org/10.1016/j.iot.2026.101932
Received 24 August 2025; Received in revised form 10 February 2026; Accepted 21 March 2026
Available online 27 March 2026
2542-6605/© 2026 The Author(s). Published by Elsevier B.V. This is an open access article under the CC BY license (http://creativecommons.org/licenses/by/4.0/).

AI is a must-have to transfer specialists' knowledge to machines, removing the work that can be automated by people and allowing people to focus on more important tasks.

The challenge in developing AI systems is the design of simulated environments that are approximated to reality, where algorithms can be trained to be further deployed in real networks to be effective.

Recent developments of Reinforcement Learning (RL) align with the growth of Large Language Models (LLMs) applications, present more innovation regarding Cybersecurity [5,6]. These recent novelties regarding AI caught the attention of experts to the use of AI to conduct offensive attacks on IT systems, as well as to attack AI itself [7], especially Reinforcement Learning. The use of AI for Cybersecurity allows systems to learn in a safe environment, rules and a decision-making process that can, after several training sessions, be transferred to real-world environments and improve the hacking process. It can impact the process by optimising workflows, finding patterns and improving decision-making processes.

Beyond RL, recent progress in Deep Learning (DL) also demonstrates the relevance of AI for security domains. Convolutional Neural Networks (CNNs) have been used to enhance the security of medical images through encryption-integrated architectures [8], deep learning workloads have been applied to forecast resource usage in cloud datacenters [9], and automated VAR systems in sports rely on deep models to support decision-making [10]. These examples show that deep neural networks can operate in complex, high-risk environments, reinforcing the potential of AI techniques to support or automate cybersecurity tasks.

Existing literature on autonomous PT mainly covers two domains: long-standing traditional tools developed from the late 1990s onward, and recent RL-based simulators that function within tightly controlled environments. Although previous reviews recognise the rapid development of these simulators, they offer limited analysis of their internal design constraints, how well they correspond to real attack surfaces, or the extent to which they can adapt to scenarios not explicitly modelled during setup. Several recurring limitations appear across the surveyed work: most simulators provide no emulation support; reward mechanisms are often manually defined and misaligned with realistic attacker strategies; current systems lack any form of continuous or incremental learning; scalability remains an issue when moving beyond small test networks; and explainability is rarely addressed, making it difficult for security teams to assess or validate the agent's decisions.

This article addresses these limitations by providing: (i) a unified comparison between traditional Cybersecurity tools and recent AI-based PT environments, (ii) a deeper analysis of newly emerged RL simulators, detailing their capabilities and technical constraints, and (iii) an examination of the remaining gaps that must be solved for these systems to operate in realistic vulnerability-assessment workflows. Building on previous work comparing conventional and AI-driven hacking tools [11], this study clarifies what current AI systems can deliver, where they fall short, and how they may be effectively integrated into future PT processes.

The study is organised as follows: Section 2 introduces the background of those two fields, and Section 3 introduces the Hacking process before and after AI. Within this section, Section 3.1 introduces the existing tools for Cybersecurity, providing details on each and discussing the problems and challenges of those. Section 3.2 introduces the AI tools being developed for this use case. Section 3.3 compares the different tools regarding the support of simulation and emulation. Section 4 discusses how hacking is being changed, a benchmark of current RL environments is discussed in Section 4.1, and, in Section 4.2, we discuss ethical concerns and identify what remains a challenge for feature work; Section 5 concludes the work presented.

2. Background

The concept of Hacking and Penetration testing is changing over time. The term Hacker started as a good thing in the 90s. It was someone who is defined as a skilled person in computers and programming, who focuses on security [12]. Over the years, with the impact of some of those actions and with the influence of the media, this term has been related to malicious people who use their skills to provoke damage or chaos. This raised the need to differentiate the hackers according to their values. The authors in Jaquet-Chiffelle and Loi [12] explain that this concept of Hacker can be divided into three categories: White hackers as someone who follow ethical principles; black hackers the ones who are malicious and perform illegal actions; and grey hackers, who are in the middle, they don't have bad intentions but follow illegal ways to obtain something.

The innovations in the field of Cybersecurity and Technology in general have led to advancements that simple tools for Penetration Testing turn now been turned into AI-based automated tool that provides a report of the vulnerabilities found [13]. Penetration Testing is a security evaluation process made by an ethical hacker [14]. Farah et al define the concept of Penetration testing as a risk assessment process to check network safety. Which, from a Cybersecurity point of view, is described as finding a way to exploit a vulnerability or making a sequence of attack chains resulting in a multi-step attack [13].

Regarding the traditional process of PT, it may vary according to the organisations and teams. Although this isn't a rule-based process, some standards are referenced in this field. The National Institute of Standards and Technology (NIST) published a Special Publication 800-115 providing guidelines for planning and assessing the PT process. IT not only focuses on the process of PT, where they highlight the four phases: planning, discovery, attack and reporting, and they provide guidelines for the post-findings process [15].

In the past years, several frameworks have been created in order to better characterise the attacks according to their objective, damage, or technique. From those MITRE ATT&CK [16] and Advanced Red Teaming (ART) stand out.

MITRE ATT&CK categorises the attacks into 3 groups: enterprise, mobile and Industrial Control Systems (ICS). Regarding the most impacted group - enterprise [2], the attacks can be categorised into 14 subgroups: Reconnaissance, Resource Development, Initial Access, Execution, Persistence Privilege Escalation, Defence Evasion, Credential Access, Discovery, Lateral Movement, Collection, Command and Control, Exfiltration and Impact. Each subgroup has its techniques that can vary according to the operating system, versions of systems, type of vulnerability, and others. For these different categories, different tools can be used to find vulnerabilities.

The problem is that there is no software that makes use of them all, combines outputs, and already defines which action/tool to use next to exploit that vulnerability, among other things.

Although the MITRE ATT&CK is seen as a theoretical framework, there is an open source project called Atomic Red Team, which is based on the MITRE framework, according to its categories, and includes for each category a set of techniques and attacks that can be used for vulnerability assessment [17].

3. Hacking before and after the use of AI

Although the Hacking process is a process where the person who is executing must follow certain actions, it consists of the use of tools that help, in some way, the person get some relevant information. This increases work productivity and offers external knowledge that the person doesn't need to fully understand to use a specific tool. Most of these tools are open source and built from the community, while more recent ones that also use AI are commercial products or subscriptions. In Sections 3.1 and 3.2, the discussion covers well-known tools and newer AI-driven solutions for vulnerability assessment. Section 3.3 compares the tools presented in the previous subsections regarding the support of simulation and emulation.

3.1. Popular tools for vulnerability assessment

With the developments of IT services, new products/systems are created, and with that, new vulnerabilities are being recognised. For that reason, new tools are used for vulnerability assessment. Recent studies show that the number of vulnerabilities is increasing, and also their capabilities [18–20]. Examples of that, and according to the MITRE framework, are:

- **Reconnaissance tools** like Shodan and Maltego enable researchers to identify internet-connected devices and analyse data relationships. Shodan, launched in 2009, is widely recognised for its ability to search for exposed devices, while Maltego, introduced in 2008, specialises in gathering intelligence for investigative purposes.
- **Resource development tools**, such as Cobalt Strike (2012) and Metasploit (2003), assist red teams in simulating threat actor activities. Cobalt Strike is a commercial-grade software for creating realistic attack scenarios, while Metasploit is a versatile open-source framework for exploitation and vulnerability assessment.
- **Initial access** tools like SET and OpenVAS help assess how attackers gain access. SET, introduced in 2009, focuses on phishing and social engineering, while OpenVAS provides comprehensive scanning capabilities to identify system vulnerabilities.
- **Execution tools**, including Empire and Covenant, help simulate malicious actions post-compromise. Empire (2015) automates PowerShell scripting for Windows exploitation, while Covenant (2018) provides a modern .NET-based framework for executing payloads.
- **Persistence tools** like Autoruns and WinPEAS help locate backdoor mechanisms. Autoruns, one of the earliest tools (1996), identifies startup programs, whereas WinPEAS (2019) focuses on privilege persistence in Windows environments.
- **Privilege escalation tools** like LinPEAS (2019) and PrivescCheck (2018) specialise in detecting exploitable configurations. LinPEAS targets Linux privilege flaws, while PrivescCheck identifies Windows vulnerabilities.
- **Defence evasion tools** such as Veil and Obfuscator.io focus on bypassing detection mechanisms. Veil (2013) generates undetectable payloads, while Obfuscator.io (2016) obfuscates scripts to prevent reverse engineering.
- **Credential access tools** like Mimikatz and Hashcat help simulate attacks on credentials. Mimikatz (2007) extracts sensitive information from memory, while Hashcat (2010) performs powerful brute-force attacks.
- **Discovery tools**, such as Nmap (1997) and BloodHound (2016), assist in exploring network environments. Nmap identifies open ports and services, while BloodHound analyses Active Directory configurations.
- **Lateral movement tools** like PsExec (2001) and Impacket (2008) allow remote command execution and protocol exploitation. PsExec enables remote process management, while Impacket supports lateral operations in networks.
- **Collection tools** like Wireshark (1998) and FileZilla (2001) facilitate data capture and transfer. Wireshark inspects network packets, while FileZilla manages file exchanges securely.
- **Command and control tools**, including Metasploit and Cobalt Strike, simulate malicious control scenarios. Metasploit (2003) creates custom channels, and Cobalt Strike (2012) emulates advanced infrastructure.
- **Exfiltration tools** such as Rclone (2012) and WinSCP (2000) help simulate data transfer scenarios. Rclone focuses on cloud transfers, while WinSCP secures data exchanges using FTP/SFTP protocols.

This wide range of tools is described as a problem from the authors [19] perspective. They conclude their work by mentioning the concern that when a specialist wants to implement one of those, they become confused about which should it use due to the high variety. Another interesting finding from the paper [20] is that sometimes specialists make use of different tools simultaneously and in an autonomous way, which causes an interruption of systems that weren't supposed to happen. This event can impact not only the Penetration Testing analysis but also the normal function of the service being analysed. These problems raise the need for further development regarding the use of these tools.

The discussed tools can be easily compared in Table 1.

3.2. AI-based tools for vulnerability assessment

Artificial Intelligence is the field that joins Maths and Computing to create models that can predict, categorise, label and make decisions based on data. This area is subdivided into Supervised Learning, Unsupervised Learning, and Reinforcement Learning [46].

Table 1
Most popular tools for vulnerability assessment.

MITRE ATT&CK subcategories	Tool Name	Release Year	Official Website/GitHub	Open Source
Reconnaissance	Shodan	2009	[21]	No
	Maltego	2008	[22]	No
Resource Development	Cobalt Strike	2012	[23]	No
	Metasploit Framework	2003	[24]	Yes
Initial Access	Social-Engineer Toolkit (SET)	2009	[25]	Yes
	OpenVAS	2009	[26]	Yes
Execution	Empire (PowerShell Empire)	2015	[27]	Yes
	Covenant	2018	[28]	Yes
Persistence	Autoruns for Windows	1996	[29]	No
	WinPEAS	2019	[30]	Yes
Privilege Escalation	LinPEAS	2019	[31]	Yes
	PrivescCheck	2018	[32]	Yes
Defense Evasion	Veil Framework	2013	[33]	Yes
	Obfuscator.io	2016	[34]	No
Credential Access	Mimikatz	2007	[35]	Yes
	Hashcat	2010	[36]	Yes
Discovery	Nmap	1997	[37]	Yes
	BloodHound	2016	[38]	Yes
Lateral Movement	PsExec	2001	[39]	No
	Impacket	2008	[40]	Yes
Collection	Wireshark	1998	[41]	Yes
	FileZilla	2001	[42]	Yes
Command and Control	Metasploit Framework	2003	[24]	Yes
	Cobalt Strike	2012	[23]	No
Exfiltration	Rclone	2012	[43]	Yes
	WinSCP	2000	[44]	No
Impact	Metasploit Framework	2003	[24]	Yes
	DiskCryptor	2008	[45]	Yes

Supervised Learning is being used for Cybersecurity in order to classify data in categories [47]: spam or not spam, virus or not virus, attack traffic or benign. Unsupervised learning works with unlabeled data and focuses on grouping items that share similar features or lie close to each other [46]. It is used for clustering files, passwords, traffic data, and more. These systems have been widely studied but are limited to what they know - the dataset used for training. For that reason and because the world is a lot more dynamic, Reinforcement Learning is gaining its space and is used as much as the other two techniques.

Researchers aim to use the Reinforcement Learning technique to conduct this Pentesting [48,49], which consists of an agent that will interact with an environment, considering its observations and possible actions, and decide which actions lead to the greatest rewards. The agent will learn a function called a policy that will maximise these rewards over time [50]. Translating this to the Penetration Testing scenario, the observations will be the exploits in the machines or systems, and the actions refer to possible attacks. If an attack is successfully made, the reward will be positive [51].

known terms introduced by Sutton et al. [52] when implementing an RL problem are:

- Environment: a simulator with rules or mechanisms that allow an agent to interact with the observations gathered by acting with the possible actions available. It can be fully observable or partially observable, referring to the agent's point of view.
- Reward: a value or points returned by the environment corresponding to the agent's action. It is commonly referred to as R_t, meaning the reward at time step t. The next reward is denoted R_{t+1}, received after the agent takes an action at time t
- State: It is commonly referred to as S_t, meaning the state at time step t. The next state is denoted S_{t+1}, which the agent transitions to after taking an action at time t.
- Episode: a sequence of interactions between an agent and the environment.
- Policy: a function that determines which action the agent will take in a certain state. The policy is trained during experiments in order to achieve the maximum possible cumulative reward (maximum rewards in an episode).

The explanatory definitions can be graphically visible in Fig. 1.

A representative example is the Cyberwheel simulator [53], an open-source RL environment released in 2024 that models pentesting over networked systems. Cyberwheel supports both single and multi-agent settings, enabling adversarial red-blue interactions as well as task-specific attackers. Its action space includes scanning, exploitation, resource management, defensive actions, and countermeasures, implemented through structured payloads mapped to concrete CVEs.

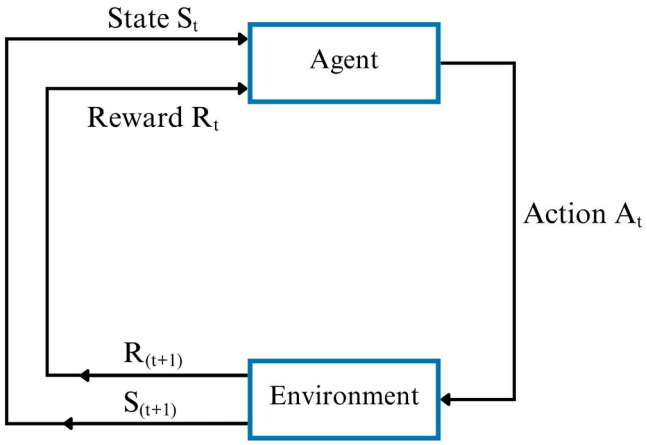


Fig. 1. Reinforcement learning architecture.

The observation space exposes network state, vulnerabilities, active defences, device properties, and attack success metrics, while rewards are driven by mission success and control objectives. Although Cyberwheel exemplifies the dominant RL-based pentesting design, alternative environments such as CybORG focus more explicitly on connection analysis, attack paths, and detection-aware red-teaming. Recent surveys show a rapid growth of such simulators, with increasing feature diversity [54].

Table 2 is the result of their comparison, removing the state column, which is not useful for this research, but it also added a new column that matches the simulators to the MITRE subgroups of attack types.

Regarding the MITRE ATTA&CK Type for each row, there is a subgroup category that highlights the others. Reconnaissance techniques exist in all of the simulators. Although VirusTotal and Hayami are significantly different from the others due to their use cases, they also use Reconnaissance techniques. VirusTotal uses reconnaissance when comparing the URL inserted or the file with an already known virus. Hayami used it by trying existing XSS payloads to validate that the security existed. Regarding Resource development, it is also present in all of them. The idea of all the simulators is to try to establish resources they can use to support operations, so all the simulators create this type of resource. Initial access is common according to the simulators, although in VirusTotal, these techniques don't exist due to the fact that in these simulators, there is no login or any access, it is just the validation of a file/URL. Execution exists in every simulator due to the fact that there are scripts that are executed and also files/URLs that contain malicious code/instructions. Regarding Persistence, the only simulator that doesn't support this is VirusTotal because it only validates a file/URL. The case of Hayami can use this technique due to the fact that if the success of payload, the code can persist in the server. For the same motive, Privilege Escalation doesn't exist in VirusTotal. Defence Evasion is an interesting detail that is used in some simulators, but especially in VirusTotal, because the main idea of this simulator is to build files/URLs that aren't detected. Credential Access is supported by CyberBattleSim, MITRE ATT&CK and Cyberwheel, which is a very specific attack vector. Discovery, similar to Reconnaissance, exists in all of the simulators. Lateral Movement is also common in the simulators, although in VirusTotal and Hayami isn't supported due to its functionality. The collection is a principal component in all the simulators due to the fact that AI uses the collection of data to improve the system itself. Command and Control aren't supported in VirusTotal. The simulator Hayami can support Command and Control when a user successfully injects XSS code that remains in the server, and then can attack future users of the web app. Exfiltration and Impact aren't supported in any of the simulators. This presents the main limitation of these simulators, which is the fact that all of them are just simulators and not yet emulators tested in real-time.

3.3. Simulators vs emulators

Although existing simulators show significant potential for defending against hackers, they have notable flaws. Currently, most of them remain as testbeds or are still under development and function solely as simulators. Simulators may yield high scores, but when applied in real-world scenarios, they can prove ineffective. The development of simulators with emulation capabilities and the ability to run tests in real-world environments is essential. In this context, cybersecurity specialists play a key role in creating environments that closely approximate real-world conditions.

Emulators are an architecture that behaves as a sandbox. It is a software-based tool that, when started, creates a network with its subnetworks. In each one, there are virtual machines (VMs) with images of different operative systems with different vulnerabilities. Then each VM sends the logs to a router or a common management software that is used for the RL agent. Those settings are in some emulators as pre-defined conditions or complexity, such as low, medium and high. In other this can be settled manually or randomly.

In one of those subnetworks, it will be a machine, most likely running a Kali Linux operating system due to its pre-installed tools used for hacking, and it will be the machine where the RL agent will be running. The described architecture is designed in Fig. 2.

Table 2
RL environments for offensive pentesting.

Simulator	Action Space	Observation Space	Reward	MITRE ATT&CK Type
CyberBattleSim [55] (open source)	Local attack Remote attack Authenticated connection	Discovered Owned nodes Discovered credentials Escalation levels Available attacks	Intrinsic node value	Initial Access Persistence Privilege Escalation Credential Access Lateral Movement Command and Control
CybORG [56] (open source)	Scanning Exploitation Privilege escalation Lateral movement Remediation	Discovered nodes Active connections System state Attack paths Available countermeasures	Mission success Penalty for detection Cumulative node control or network dominance	Initial Access Persistence Privilege Escalation Defense Evasion Lateral Movement Command and Control
NASim [57] (open source)	Scanning Exploitation Privilege escalation Lateral movement	Discovery nodes Owned nodes Nodes vulnerabilities Network topology	Cumulative rewards based on success exploits Penalty for detection Node value	Initial Access Persistence Privilege Escalation Defense Evasion Lateral Movement Command and Control
NASimEmu [58] (open source)	Scanning Exploitation Privilege escalation Lateral movement	Discovery nodes Owned nodes Nodes vulnerabilities Network topology	Positive reward when the agent gains privilledge access	Initial Access Persistence Privilege Escalation Defense Evasion Lateral Movement Command and Control
MITRE ATT&CK and CALDERA [59] (partially open source)	Scanning Exploitation Privilege escalation Lateral movement Persistence Credential dumping	Discovered nodes Access privileges Active defenses Technique success/failure System logs	Mission success penalty for detection System compromise Persistence achievement	Initial Access Persistence Privilege Escalation Defense Evasion Credential Access Lateral Movement Command and Control
VirusTotal [60] (public online application)	Uploading files Scanning URLs Generating reports	Detected malware Threat scores Network Indicators Metadata	Threat detection Aggregated threat scores	Collection Defense Evasion
MulVAL [61] (open source)	Scanning Exploitation Privilege escalation Lateral movement Attack graph generation	Discovery nodes Vulnerabilities Attack paths Network topology Exploit success rates	Successful completion of attack paths Node value Penalty for detection Risk evaluation	Initial Access Persistence Privilege Escalation Defense Evasion Lateral Movement Command and Control
PenBox [62] (open source)	Scanning Exploitation Privilege escalation Attack validation Post-exploitation actions	Discovered hosts Vulnerabilities Exploit results System state	Successful exploitation Privilege level Access to sensitive data Test coverage	Initial Access Persistence Privilege Escalation Lateral Movement Command and Control
Hayami [63] (partially open source)	Generating attack payloads Sending requests Analysing responses Adapting payloads	Detected vulnerabilities Response codes Exploit success Application feedback	Successful Vulnerability discovery Efficiency in finding XSS Payload effectiveness	Initial Access Persistence Privilege Escalation Collection Command and Control
PenGym [64] (open source)	Scanning Exploitation Privilege escalation Lateral movement Reconnaissance	Discovered vulnerabilities Network topology Exploit results Host information	Successful exploits Discovery of high-value targets Lateral movement success	Initial Access Persistence Privilege Escalation Lateral Movement Command and Control
Cyberwheel [53] (open source)	Scanning Exploitation Resource management Defensive actions Countermeasures deployment	Network state Active defenses Vulnerabilities Attack/Defend success rates	Mission success Attack penetration Defense stability Strategic control of resources	Initial Access Persistence Privilege Escalation Credential Access Defense Evasion Lateral Movement Command and Control

Cyberwheel simulator presents its architecture named Firewheel that uses Kernel-based Virtual Machine (KVM) to virtualise host machines and Minimega[1] to launch and manage VMs [53] aligned to Ansible[2] that autonomously deploys Firewheel. To collect the logs from the VMs, the Sysmon software[3] is used. In their architecture, one of those VMs is responsible for running a Security Information and Event Management (SIEM) solution with the Elastic Stack[4] tool. The logs generated and gathered by the SIEM are then converted into an observation space vector to be used by the agent.

NASimEmu also makes use of Vagrant to manage the network of VMs and uses VirtualBox for creating the VMs using configurable Metasploitable images [58]. The attacker machine runs Kali Linux with Metasploit [24] that is remotely connected to the NASimEmu interface. When the agent decides the action to take, it is translated to a command for the Metasploit framework.

Huynh et al present PenGym [65] as a framework for training pentesting RL agents in realistic network environments, comparing its results with the NASim [57]. Although this idea is not discussed in other papers, due to this article being a novelty, it already shows interesting insights. When testing agents trained in different scenarios of complexity, PenGym outperforms NASim while requiring more training time due to the synchronisation of the feedback and decision-making.

[1] https://www.sandia.gov/minimega/
[2] https://www.redhat.com/en/ansible-collaborative
[3] https://learn.microsoft.com/en-us/sysinternals/downloads/sysmon
[4] https://www.elastic.co/elastic-stack/

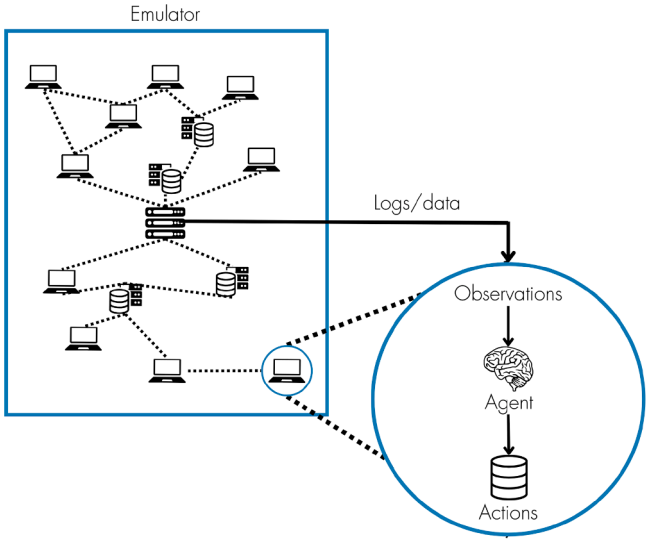


Fig. 2. Emulator architecture.

These emulators, although configurable, meet scalability problems. An interesting approach presented by Cyborg [56] is the use of the cloud. The authors used Amazon Web Services (AWS) with VMs to create their environment. They use the IaaS to take advantage of automatically set up IPs for each subnet and host, and the easy setup or shutdown of VMs. They also used third-party tools such as the Metasploit framework and Velociraptor[5] that behave as a digital forensic and incident response tool by collecting and analysing logs and data.

4. Discussion

Vulnerability Assessment tools identified in Section 3.1 give us a wide perspective of which tools exist for each MITRE ATT&CK subcategories. It also shows that most of them were developed more than 10 years ago. While the open sources have been maintained by the community, others may have stopped their development.

Some of the traditional tools now play an important role in the RL-based tools. Those are used as an available action that an RL agent can take. The challenge sometimes is the way to use it as an action. NaSimEmu [58] as an example, develop a wrapper for Metasploit [24] to call its command instructions in a Python program. The more tools used, the more wrappers of tools have to be made. The idea of using Metasploit is due to the fact that it already has built-in payloads and tools for attacks.

The evolution of the VA process seen in Table 2 shows that AI, especially Reinforcement Learning, is taking an important role in the development of new tools. This is transforming the way VA is done. The idea of having an agent that is trained for Pentesting is promising, but there are some limitations that still need to be addressed.

An important consideration is the outdated data. These agents perform well for attacks and CVEs known during training; however, as new cyberattacks evolve, the agents become outdated. This necessitates rethinking how to make the agents smarter and capable of lifelong learning.

While these agents function as all-in-one tools, specific techniques outlined in the MITRE framework are not yet supported. Notably, Exfiltration and Impact are not properly implemented, likely due to the experimental stage of these simulators. This limitation underscores the need for further development to ensure comprehensive coverage of adversarial tactics and techniques, particularly those critical for assessing real-world cybersecurity threats.

We highlight that the major challenge in these RL environments remains the ability to transfer the agent's knowledge to real scenarios. The lack of emulators identified in Section 3.3 highlights the problem we are facing. There is a need for development environments that support both simulation and emulation. This will allow researchers to access the agent's knowledge in real-world tests.

The initial research conducted in Fernandes et al. [11] is now extended to cover the other recent RL environments found in Table 3. Two more columns have been added to compare the environments against themselves: Simulation and Emulation support.

Table 3 shows that, comparing the Traditional Tools with the RL-based tools, it is notable that the use of AI and simulation is present in all the RL-based tools. The ability to simulate the use of techniques of the RL agent is an important step, so we can observe the agent and network behaviour. While simulation is present in all the RL-based tools, we can see that only four of them support emulation.

[5] https://docs.velociraptor.app/

Table 3
Comparison of RL environments for pentesting.

Tool	Offensive/Defensive Actions	Use of AI	Simulation	Emulation	Open source
Traditional Tools	One or another				
CyberBattleSim [55]		✓	✓		✓
CybORG [56]	Both	✓	✓	✓	✓
NASim [57]		✓	✓		✓
NASimEmu [58]		✓	✓	✓	✓
Caldera [59]	Both	✓	✓	✓	partially
VirusTotal [60]		✓	✓		
MulVAL [61]		✓	✓		✓
PenBox [62]		✓	✓		✓
Hayami [63]		✓	✓		partially
PenGym [64]		✓	✓	✓	✓
Cyberwheel [53]	Both	✓	✓		✓

The fact that emulation is an ongoing development for some environments and a next goal for others reveals that scalability isn't being discussed. The creation of an emulated network environment of an industry would result in a series of virtual machines and networks. It can be computationally expensive and might create the need for heavy machines to be able to test the different scenarios. The topic of scalability should also be considered as a future research direction for the development of RL-based tools for Pentesting.

Regarding the support of Offensive and Defensive actions, three of the eleven RL-based tools support both, which means that we can train agents for a specific team or even, in some cases, we can train two agents that learns with each other, creating a gamification of RL agents, improving both simultaneously.

4.1. Environments benchmark

The wide range of environments raises the importance of evaluating and benchmarking them across different dimensions. The conducted benchmark uses five dimensions that reflect how close each environment is to practical, large-scale autonomous PT:

- *Environment fidelity*, which measures how closely the simulator approximates real hosts, services and network behaviour (from abstract graphs to full VM-based emulation);
- *Attack coverage*, which reflects how many MITRE ATT&CK tactics and techniques are supported end to end;
- *Scalability*, which captures the ability to represent larger and more complex topologies without prohibitive resource costs;
- *Operational complexity*, which indicates the effort required to install, configure, orchestrate and maintain the environment; and
- *Documentation & maintainability*, which evaluates the availability of documentation, examples and ongoing community or vendor support.

Across the evaluated environments, the emulation-based systems exhibit the highest environment fidelity. NASimEmu achieves the strongest score because it executes real VM images and Metasploit payloads, providing concrete host behaviour and realistic attack effects. Caldera also achieves high fidelity through adversary-emulation plugins mapped to ATT&CK techniques. PenGym and Cyberwheel, although not full emulators, still achieve high fidelity because they incorporate realistic network states, updated CVEs, and detailed action models. In contrast, abstract graph-based simulators such as CyberBattleSim, NASim and MulVAL provide only simplified representations of networks and therefore achieve lower fidelity. VirusTotal and Hayami operate at a very narrow scope and do not model network behaviour, resulting in the lowest fidelity.

Attack coverage follows a similar trend. Caldera and Cyberwheel cover the widest range of ATT&CK tactics, followed by CybORG and PenGym, which support multiple attack stages such as lateral movement, privilege escalation and persistence. NASim, NASimEmu and CyberBattleSim cover only a subset of these behaviours, while MulVAL provides coverage limited to attack-graph generation. VirusTotal and Hayami cover only specific tasks (malware evaluation and XSS payload generation), resulting in minimal coverage.

Scalability is highest in simulation-only frameworks that rely on abstract representations. NASim and MulVAL scale easily to large topologies because they operate on lightweight graph structures. VirusTotal, being an external service, also provides effectively unlimited scalability. In contrast, NASimEmu, PenGym and Cyberwheel handle more realistic configurations but incur higher resource requirements. Emulation layers significantly reduce scalability, placing NASimEmu at the lower end of this dimension.

Operational complexity is lowest in CyberBattleSim, VirusTotal and PenBox, where installation and execution rely on lightweight abstractions or external platforms. MulVAL has moderate complexity due to its attack-graph pipeline. Caldera and CybORG require more configuration because of plugin management and multi-agent setups. NASimEmu, PenGym and Cyberwheel have the highest complexity as they incorporate VM orchestration, deployment scripts or modular strategy engines that require detailed configuration.

Documentation and maintainability reflect repository activity and ecosystem maturity. Cyberwheel, PenGym, NASimEmu and NASim are actively maintained and supported by thesis or recent publications. CyberBattleSim also has acceptable support. CybORG and Caldera have stable documentation but less frequent updates. MulVAL, PenBox and Hayami provide limited or outdated documentation, and VirusTotal's documentation is minimal because it relies almost entirely on the external web service.

Overall, the comparison shows a clear trade-off between realism and practicality. Environments that are simulator-based or are abstracted by not executing real actions are easy to scale in memory and also easy to use. While this relates to an advantage, it

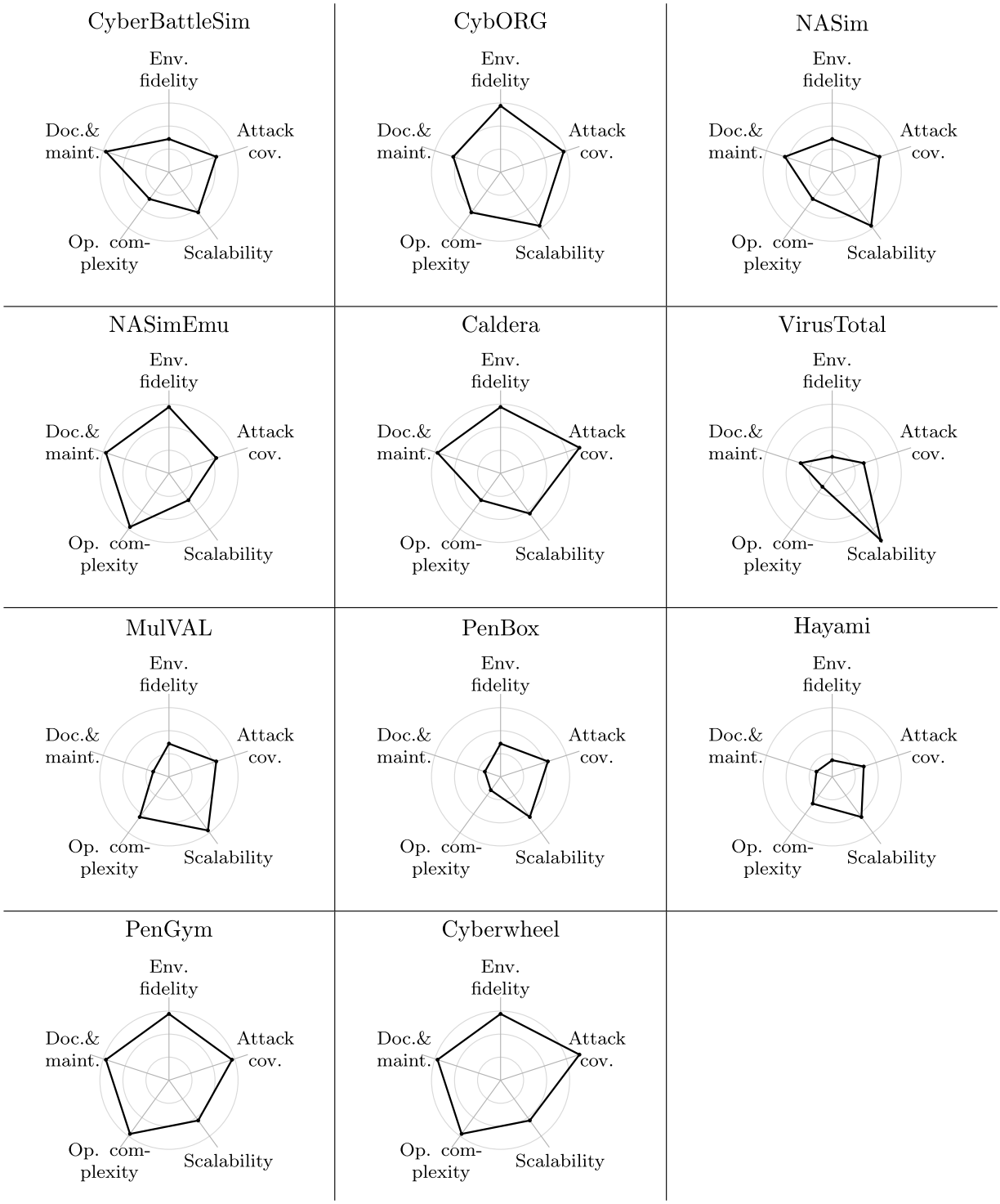


Fig. 3. Radar-chart comparison of AI-based penetration testing simulators across environment fidelity, attack coverage, scalability, operational complexity, and documentation & maintainability.

is also a disadvantage that lacks realism. Emulation-capable or high-fidelity tools allow the creation of a real behaviour but are harder to configure, less scalable and require stronger documentation to support long-term use. None of the current environments performs strongly across all five dimensions, reinforcing the need for hybrid approaches that combine scalable simulation with targeted emulation while reducing operational complexity. This benchmark comparison can be seen in Fig. 3.

Although the scenario is not ideal, it is an evolution compared to the previously seen non-AI-based tools. The use of intelligence in Vulnerability Assessment by using RL allows an increase in performance and practicality due to the autonomous decision-making by the AI.

4.2. Limitations

Relevant limitations of these RL-based tools are related to the explainability. While RL agents can be somewhat easy to understand due to their reward mechanism defined in the environment, the current environment isn't prepared to show the user the attack graph planned or a report of the actions taken and the rewards obtained in each. A final report would be beneficial for the user to understand where the entry point was and identify the vulnerabilities found.

The topic of explainability points to concerns about ethics in using AI for Pentesting. This topic hasn't been discussed in the identified papers. The variety of environments should follow some guidelines of environmental design in order to follow the best practices and ethical guidelines.

Besides the studies published and the frameworks developed, there are already some regulations regarding the use of AI. The EU AI Act[6] was developed by the European Union and published in 2024. This document consists of a legislative proposal to regulate the development and use of AI within the EU. From that document, there is a Regulatory Framework that defines four levels of risk for AI systems: unacceptable risk, high risk, limited risk, and minimal risk [66]. Unacceptable risk means the system cannot be developed or put into production. High risk means the system must meet strict obligations before it can be released to the market. Limited risk requires disclosure of the data used and the type of system deployed. Minimal risk covers all other use cases.

Focusing on the risk that AI can cause, NIST developed the AI Risk Management Framework[7] with four main actions. The governance action, where the developers must ensure roles within the development team, policies, accountability for the development of each component, asset management and others. The Mapping action related to understanding the business and the technology around the AI software, what data is used, the impacts of the use of the AI, threats and the stakeholders that can be negatively impacted. The Measure action consists of measuring the identified risks, testing the performance of the AI system, testing it according to security, robustness, privacy, fairness, explainability and other values. With those phases in mind, the Manage Action implicates the risk priorisation, application of controls, documentation and monitoring, response plan and continuous improvements. The four phases are defined as a continuous process and not a one-time process. All four should be connected, and the development team should be involved in the analysis of the framework.

Despite different frameworks and baselines to develop Ethical AI [67], some authors raise the need for translating principles into practice by using case studies and applying ethical principles to those cases, which is a future research direction within the scope of this research [68].

5. Conclusions and future work

In this study we present a comparison between traditional Cybersecurity tools and recent AI-based Penetration Testing environments. A deeper analysis of newly emerged RL simulators was done, detailing their capabilities and technical constraints, and remaining gaps were identified that must be addressed for these systems to operate in realistic vulnerability-assessment workflows.

The comparison of simulated and emulated environments revealed clear strengths in isolated dimensions: NASimEmu, Caldera, and Cyberwheel provide higher environment fidelity; Caldera, Cyberwheel, and CybORG offer broader MITRE ATT&CK coverage; and NASim, MulVAL, and CyberBattleSim scale more efficiently. These findings showed that the field is still fragmented: each environment fits a specific need, but none supports end-to-end training that generalises beyond controlled scenarios. The disconnection between simulation performance and real-world applicability reinforces that the current generation of RL tools is still insufficient for operational pentesting workflows.

As future work we propose to prioritise hybrid environments that combine scalable simulation with targeted emulation, to expand MITRE ATT&CK coverage, to standardise reward structures, and to introduce mechanisms for continuous learning. These steps are essential for enabling RL agents to adapt to unseen networks and operate reliably in practical Vulnerability Assessment settings.

CRediT authorship contribution statement

Rui Fernandes: Writing – original draft, Methodology, Investigation, Conceptualization; **Nuno Lopes:** Writing – review & editing, Supervision, Conceptualization; **Joaquim Gonçalves:** Writing – review & editing, Supervision, Conceptualization; **John Cosgrove:** Writing – review & editing, Supervision, Conceptualization.

[6] https://artificialintelligenceact.eu/

[7] https://www.nist.gov/itl/ai-risk-management-framework

Data availability

No data was used for the research described in the article.

Declaration of competing interest

The authors declare the following financial interests/personal relationships which may be considered as potential competing interests: Rui Fernandes reports financial support was provided by FCT - Fundação para a Ciência e a Tecnologia. If there are other authors, they declare that they have no known competing financial interests or personal relationships that could have appeared to influence the work reported in this paper.

Acknowledgments

This paper was funded by national funds and FCT/MCTES (PIDDAC), through the Foundation for Science and Technology, I. P. (FCT) under the scope of the project UID/PRR/05549/2025 with DOI 10.54499/UID/PRR/05549/2025 and under the scope of the project LASI-LA/P/0104/2020. It was also funded with National funding by FCT, through the individual research grant PRT/BD/154919/2023 with the corresponding DOI https://doi.org/10.54499/PRT/BD/154919/2023. This paper was also partially funded by the project “CyberPRAISE - Cybersecurity research for PRivAte, Intelligent and truStablE solutions”, (NORTE2030-FEDER-01820300), supported by the European Regional Development Fund (ERDF), through the Northern Regional Operational Program | NORTE2030, under the PORTUGAL 2030 Partnership Agreement.

References

[1] Z. Zeng, D. Huang, G. Xue, Y. Deng, N. Vadnere, L. Xie, ILLATION: Improving vulnerability risk prioritization by learning from network, IEEE Trans. Depend. Secure Comput. 21 (4) (2024) 1890–1901. https://doi.org/10.1109/TDSC.2023.3294433.
[2] E. U. A. for Cybersecurity (ENISA), Cyber Europe 2024, 2024, Accessed: 2026-02-10, https://www.enisa.europa.eu/sites/default/files/2024-12/Cyber_Europe_AAR_2024_1.pdf.
[3] S. Institute, What You Need to Know About the SolarWinds Supply Chain Attack, 2020, Accessed: 2026-02-10, https://isc.sans.edu/diary/26884.
[4] D.E. Sanger, Deepfake technology impersonates U.S. officials, prompting warnings about AI risks, The New York Time. (2024). Accessed: 2026-02-10, https://www.nytimes.com/2024/05/31/us/politics/deepfake-us-official-russia.html.
[5] J. Zhang, H. Bu, H. Wen, Y. Liu, H. Fei, R. Xi, L. Li, Y. Yang, H. Zhu, D. Meng, When LLMs meet cybersecurity: a systematic literature review, Cybersecurity 8 (1) (2025) 55. https://doi.org/10.1186/s42400-025-00361-w.
[6] F.N. Motlagh, M. Hajizadeh, M. Majd, P. Najafi, F. Cheng, C. Meinel, Large Language Models in Cybersecurity: State-of-the-Art, 2024, arXiv:2402.00891.
[7] Y. Mirsky, A. Demontis, J. Kotak, R. Shankar, D. Gelei, L. Yang, X. Zhang, M. Pintor, W. Lee, Y. Elovici, B. Biggio, The threat of offensive AI to organizations, Comput. Secur. 124 (2023) 103006. https://doi.org/10.1016/j.cose.2022.103006.
[8] P. Aruchamy, M. Sundarrajan, M.D. Choudhry, A. Jothi, Enhancing security of medical images using code-based intermittent encryption and convolutional neural network, Comput. J. 68 (5) (2024) 539–551. https://doi.org/10.1093/comjnl/bxae130.
[9] A. Prasanth, K.N. Babu, P. Rahul, B.V. Reddy, A deep learning-based workload forecasting model in cloud data centers, in: 2025 International Conference on Computing Technologies (ICOCT), 2025, pp. 1–5. https://doi.org/10.1109/ICOCT64433.2025.11118402.
[10] H.A. Zaken, A. Prasanth, M.A. Jahjah, An automated deep learning-based VAR for enhancing referee decision-making in football, in: 2025 3rd International Conference on Intelligent Data Communication Technologies and Internet of Things (IDCIoT), 2025, pp. 2247–2252. https://doi.org/10.1109/IDCIOT64235.2025.10914757.
[11] R. Fernandes, N. Lopes, J. GonÃ§alves, J. Cosgrove, Comparing traditional hacking tools and AI-driven alternatives, in: 2025 13th International Symposium on Digital Forensics and Security (ISDFS), 2025, pp. 1–5. https://doi.org/10.1109/ISDFS65363.2025.11012027.
[12] D.-O. Jaquet-Chiffelle, M. Loi (Eds.), Ethics of Cybersecurity, Springer International Publishing, Cham, 2020. https://doi.org/10.1007/978-3-030-29053-5.
[13] F. Abu-Dabaseh, E. Alshammari, Automated penetration testing: an overview, in: Proceedings of the 2018 International Conference on Computer Science and Information Technology (CSIT), Computer Science & Information Technology (CS & IT), 2018, pp. 121–129. https://doi.org/10.5121/csit.2018.80610.
[14] E.A. Altulaihan, A. Alismail, M. Frikha, A survey on web application penetration testing, Electron. (Basel) 12 (5) (2023). https://doi.org/10.3390/electronics12051229.
[15] K. Scarfone, M. Souppaya, A. Cody, A. Orebaugh, Technical Guide to Information Security Testing and Assessment, Technical Report SP 800-115, National Institute of Standards and Technology (NIST), 2008. Accessed: 2026-02-10, https://nvlpubs.nist.gov/nistpubs/Legacy/SP/nistspecialpublication800-115.pdf.
[16] MITRE, MITRE att&ck®, 2024, (

Note 1. https://attack.mitre.org/). Accessed: 2026-02-10.
[17] R. Canary, Atomic Red Team, 2024, Accessed: 2026-02-10, https://github.com/redcanaryco/atomic-red-team.
[18] D. Railkar, A study on vulnerability scanning tools for network security, Int. J. Sci. Res. Comput. Sci. Eng. Inform. Technol. 8 (2022) 340. https://doi.org/10.32628/CSEITCN228641.
[19] K. Kritikos, K. Magoutis, M. Papoutsakis, S. Ioannidis, A survey on vulnerability assessment tools and databases for cloud-based web applications, Array 3–4 (2019) 100011. https://doi.org/10.1016/j.array.2019.100011.
[20] S. Shah, B.M. Mehtre, An overview of vulnerability assessment and penetration testing techniques, J. Comput. Virol. Hack. Techniq. 11 (2015) 27–49. https://doi.org/10.1007/s11416-014-0231-x.
[21] Shodan, 2009, (https://www.shodan.io/). Accessed: 2026-02-10.
[22] Maltego, 2008, (https://www.maltego.com/). Accessed: 2026-02-10.
[23] Cobalt Strike, 2012, (https://www.cobaltstrike.com/). Accessed: 2026-02-10.
[24] Metasploit Framework, 2003, (https://www.metasploit.com/). Accessed: 2026-02-10.
[25] Social-Engineer Toolkit (SET), 2009, (https://github.com/trustedsec/social-engineer-toolkit). Accessed: 2026-02-10.
[26] OpenVAS, 2009, (https://www.openvas.org/). Accessed: 2026-02-10.
[27] Empire (PowerShell Empire), 2015, (https://github.com/EmpireProject/Empire). Accessed: 2026-02-10.
[28] Covenant, 2018, (https://github.com/cobbr/Covenant). Accessed: 2026-02-10.
[29] Autoruns for Windows, 1996, (https://docs.microsoft.com/en-us/sysinternals/downloads/autoruns). Accessed: 2026-02-10.
[30] WinPEAS, 2019, (https://github.com/carlospolop/PEASS-ng). Accessed: 2026-02-10.
[31] LinPEAS, 2019, (https://github.com/carlospolop/PEASS-ng). Accessed: 2026-02-10.
[32] PrivescCheck, 2018, (https://github.com/itm4n/PrivescCheck). Accessed: 2026-02-10.

[33] Veil Framework, 2013, (https://github.com/Veil-Framework/Veil). Accessed: 2026-02-10.
[34] Obfuscator.io, 2016, (https://obfuscator.io/). Accessed: 2026-02-10.
[35] Mimikatz, 2007, (https://github.com/gentilkiwi/mimikatz). Accessed: 2026-02-10.
[36] Hashcat, 2010, (https://hashcat.net/hashcat/). Accessed: 2026-02-10.
[37] Nmap, 1997, (https://nmap.org/). Accessed: 2026-02-10.
[38] BloodHound, 2016, (https://github.com/SpecterOps/BloodHound). Accessed: 2026-02-10.
[39] PsExec, 2001, (https://docs.microsoft.com/en-us/sysinternals/downloads/psexec). Accessed: 2026-02-10.
[40] Impacket, 2008, (https://github.com/SecureAuthCorp/impacket). Accessed: 2026-02-10.
[41] Wireshark, 1998, (https://www.wireshark.org/). Accessed: 2026-02-10.
[42] FileZilla, 2001, (https://filezilla-project.org/). Accessed: 2026-02-10.
[43] Rclone, 2012, (https://github.com/rclone/rclone). Accessed: 2026-02-10.
[44] WinSCP, 2000, (https://winscp.net/). Accessed: 2026-02-10.
[45] DiskCryptor, 2008, (https://github.com/diskcryptor/diskcryptor). Accessed: 2026-02-10.
[46] A. Handa, A. Sharma, S.K. Shukla, Machine learning in cybersecurity: a review, WIREs Data Min. Knowl. Discov. 9 (4) (2019) e1306. https://doi.org/10.1002/widm.1306.
[47] R. Fernandes, N. Lopes, Network intrusion detection packet classification with the HIKARI-2021 dataset: a study on ML algorithms, in: 2022 10th International Symposium on Digital Forensics and Security (ISDFS), 2022, pp. 1–5. https://doi.org/10.1109/ISDFS55398.2022.9800807.
[48] S. Zurowski, G. Lord, I. Baggili, A quantitative analysis of offensive cyber operation (OCO) automation tools, in: Proceedings of the 17th International Conference on Availability, Reliability and Security, ARES '22, Association for Computing Machinery, New York, NY, USA, 2022. https://doi.org/10.1145/3538969.3544414.
[49] Y. Cheng, B. Cui, C. Chen, T. Baker, T. Qi, Static vulnerability mining of IoT devices based on control flow graph construction and graph embedding network, Comput. Commun. 197 (2023) 267–275. https://doi.org/10.1016/j.comcom.2022.10.021.
[50] V. Mnih, K. Kavukcuoglu, D. Silver, A.A. Rusu, J. Veness, M.G. Bellemare, A. Graves, M. Riedmiller, A.K. Fidjeland, G. Ostrovski, S. Petersen, C. Beattie, A. Sadik, I. Antonoglou, H. King, D. Kumaran, D. Wierstra, S. Legg, D. Hassabis, Human-level control through deep reinforcement learning, Nature 518 (7540) (2015) 529–533. https://doi.org/10.1038/nature14236.
[51] J. Schwartz, H. Kurniawati, Autonomous Penetration Testing using Reinforcement Learning, 2019, arXiv:1905.05965.
[52] R.S. Sutton, A.G. Barto, Reinforcement Learning: An Introduction, MIT Press, 2nd edition, 2018. http://incompleteideas.net/book/the-book-2nd.html.
[53] S. Oesch, A. Chaulagain, B. Weber, M. Dixson, A. Sadovnik, B. Roberson, C. Watson, P. Austria, Towards a high fidelity training environment for autonomous cyber defense agents, in: Proceedings of the 17th Cyber Security Experimentation and Test Workshop, CSET '24, Association for Computing Machinery, New York, NY, USA, 2024, p. 91–99. https://doi.org/10.1145/3675741.3675752.
[54] R. Fernandes, N. Lopes, J. Gonçalves, J. Cosgrove, Autonomous Pentesting Using Reinforcement Learning: A Systematic Literature Review, 2025, (SSRN Electron. J.). https://doi.org/10.2139/ssrn.5208526.
[55] T. Microsoft Defender Research, CyberBattleSim, 2021, (https://github.com/microsoft/cyberbattlesim). Created by Christian Seifert, Michael Betser, William Blum, James Bono, Kate Farris, Emily Goren, Justin Grana, Kristian Holsheimer, Brandon Marken, Joshua Neil, Nicole Nichols, Jugal Parikh, Haoran Wei.
[56] C. Baillie, M. Standen, J. Schwartz, M. Docking, D. Bowman, J. Kim, Cyborg: An autonomous cyber operations research gym, (2020). arXiv preprint arXiv:2002.10667.
[57] J. Schwartz, H. Kurniawatti, NASim: Network Attack Simulator, 2019, (https://networkattacksimulator.readthedocs.io/).
[58] J. Janisch, T. Pevný, V. Lisý, NASimEmu: network attack simulator & emulator for training agents generalizing to novel scenarios, in: Computer Security. ESORICS 2023 International Workshops, Springer Nature Switzerland, Cham, 2024, pp. 589–608.
[59] D. Miller, R. Alford, A. Applebaum, H. Foster, C. Little, B. Strom, Automated Adversary Emulation: a Case for Planning and Acting with Unknowns, MITRE Corp McLean VA McLean (2018).
[60] VirusTotal, VirusTotal, 2024, (https://www.virustotal.com). Accessed: 2026-02-10.
[61] X. Ou, W.F. Boyer, M.A. McQueen, A scalable approach to attack graph generation, in: Proceedings of the 13th ACM Conference on Computer and Communications Security, CCS '06, Association for Computing Machinery, New York, NY, USA, 2006, p. 336–345. https://doi.org/10.1145/1180405.1180446.
[62] M. Wallum, Streamlining Security Testing and Security Risk Management as Part of a Secure System Engineering Framework at ESA, Technical Report, European Space Agency (ESA), 2019. Ground System Architectures Workshop (GSAW), Published by The Aerospace Corporation.
[63] R. Suzuki, N. Fukuta, Prototyping an agent for dynamic generation of attack-payloads in web application vulnerability assessment, in: 2023 15th International Congress on Advanced Applied Informatics Winter (IIAI-AAI-Winter), 2023, pp. 37–44. https://doi.org/10.1109/IIAI-AAI-Winter61682.2023.00016.
[64] N. Huynh Phuong Thanh, Z. Chen, K. Hasegawa, K. Fukushima, R. Beuran, Pengym: pentesting training framework for reinforcement learning agents, in: ICISSP, 2024, pp. 498–509. https://doi.org/10.5220/0012367300003648.
[65] H.P.T. Nguyen, K. Hasegawa, K. Fukushima, R. Beuran, Pengym: realistic training environment for reinforcement learning pentesting agents, Comput. Secur. 148 (2025) 104140. https://doi.org/10.1016/j.cose.2024.104140.
[66] E. Union, Regulation (EU) 2024/1689 of the European Parliament and of the Council of 27 September 2024, 2024, Accessed: 2026-02-10, https://eur-lex.europa.eu/legal-content/EN/TXT/?uri=CELEX:32024R1689.
[67] L. Floridi, J. Cowls, A unified framework of five principles for AI in society, Harvard Data Sci. Rev. 1 (1) (2019). https://hdsr.mitpress.mit.edu/pub/l0jsh9d1.
[68] B. Mittelstadt, Principles alone cannot guarantee ethical AI, Nat. Mach. Intell. 1 (2019) 501–507. https://doi.org/10.1038/s42256-019-0114-4.